

CSCE 701 CYBERSECURITY · MODULE 06 · LECTURE NOTES

Web Security

The browser's security model, and the classes it does and does not cover.

MODULE OUTCOMES

- Explain the same-origin policy and what an origin is.
- Explain the injection classes and the one correct defence.
- Explain CSRF and the modern mitigations.
- Explain the security headers and what each does.
- Review a web application against the standard classes.

1 The origin

THE SAME-ORIGIN POLICY IS THE BROWSER'S ONE SECURITY BOUNDARY

An origin is the triple (scheme, host, port). Script running in one origin cannot read responses from another — which is the entire basis of web security, and everything else is a refinement of or an exception to it.

Note carefully what it does *not* prevent: a page can freely *send* requests to any origin — images, stylesheets, scripts, and form submissions all load and post cross-origin by design, because the web was built on that — **and it is only *reading the response* that is blocked**.

Which is exactly why CSRF exists (§3): the attacker's page can cause a request that carries your cookies, and cannot read the answer — and for a state-changing request, causing it is entirely sufficient.

And CORS *relaxes* the policy rather than enforcing it. A permissive CORS configuration is a hole that you opened, not a control that you added — which is a common and severe misconfiguration precisely because **the direction of the mechanism is counterintuitive**: developers add CORS headers to make something work and do not read them as granting access.

2 Injection

THE CAUSE, EVERY TIME: data is concatenated into a string which is then PARSED AS CODE by some interpreter. The parser cannot tell your data from your structure, because by the time it sees them they are the same string.

SQL injection	-> the database's SQL parser
command injection	-> the shell's parser
cross-site script.	-> the browser's HTML and JS parsers
LDAP, XPath, NoSQL	-> their respective query parsers
template injection	-> the template engine's parser
log injection	-> whatever later reads the log

THE DEFENCE THAT WORKS: never build the string. Use an interface that keeps data and structure SEPARATE all the way down.

SQL	-> parameterised queries. Always, everywhere.
shell	-> pass an argument array rather than a string; better still, do not invoke a shell at all
HTML	-> contextual escaping by the template engine, or a DOM API that takes text rather than markup
JSON	-> a serialiser, never string building

THE DEFENCES THAT DO NOT WORK RELIABLY

- blocklists of dangerous characters -- you will miss an encoding, and there is always another encoding
- input sanitisation at the boundary -- the CORRECT escaping depends on the DESTINATION context, and the input boundary does not know the destination
- escaping by hand -- correct until someone changes the context around it, which nobody notices

SO: ESCAPE AT THE POINT OF USE, BY CONTEXT, USING THE INTERPRETER'S OWN PARAMETERISATION.

'Escape at the point of use, by context' is the sentence that changes behaviour, and the reason is in the third non-defence: **the correct escaping depends on where the value lands, and the input boundary cannot know where that will be. Sanitising on input feels safer and is structurally unable to be correct**, which is why frameworks moved to contextual auto-escaping on output.

CROSS-SITE SCRIPTING IS INJECTION INTO THE BROWSER, AND CONTEXT IS EVERYTHING

The same value requires different escaping depending on where it lands: as HTML element text, as an attribute value (quoted or unquoted), inside a script block, inside a URL, or inside CSS — five different escaping rules, and the wrong one is frequently worse than none because it looks safe.

Which is why a single escapeHtml function is insufficient — it is correct for element text, wrong inside an unquoted attribute, and actively dangerous inside a script block, where HTML escaping does nothing useful and JavaScript string escaping is what is needed.

So use a template engine that escapes by context automatically (which the major modern ones do), and treat any manual escaping, any innerHTML assignment, and any escape-bypass directive as a finding that must be individually justified — which makes the review tractable, because the list of bypasses is short and greppable.

And Content Security Policy is the defence in depth (§4) — it does not fix the injection and it substantially limits what an injection can accomplish, which is Module 01 §3's layering applied to a specific class.

3 CSRF

CSRF: THE BROWSER ATTACHES YOUR CREDENTIALS TO A REQUEST THE ATTACKER CAUSED

An attacker's page submits a form to your site, or loads an image whose URL is a state-changing endpoint. The browser attaches your session cookie, because that is what cookies do. Your server acts on your authority. Module 03 §2's confused deputy, exactly — with the browser as the deputy.

The classical mitigation is a synchroniser token: an unpredictable value bound to the session, required on every state-changing request, which the attacker's page cannot read because of the same-origin policy (§1) — so the defence rests on exactly the property the attack exploits the absence of.

And the modern mitigation is the SameSite cookie attribute — Lax is now the default in major browsers, which breaks the basic cross-site form submission without any application change at all.

So the current position is: rely on SameSite as the baseline, add synchroniser tokens for anything sensitive, never use GET for a state change (because GET requests are triggered by images and prefetching), and verify the Origin header on state-changing requests. Defence in depth, because the cookie default can be overridden by a developer who needed cross-site posting for something else.

4 The headers, and the review

Header	What it does	Note
Content-Security-Policy	Restricts which sources may be loaded and which script may execute.	The highest-value header, and genuinely hard to deploy — see the note.
Strict-Transport-Security	Forces HTTPS for this host for a stated period, including the very first visit if preloaded.	Cheap and effective — it closes the downgrade window that Module 04's TLS cannot.
X-Content-Type-Options: nosniff	Stops the browser guessing a content type different from the declared one.	One line, and it prevents a real vulnerability class where an uploaded file is interpreted as script.
X-Frame-Options, or CSP frame-ancestors	Prevents your page being framed — which is clickjacking.	Use the CSP directive, which is the modern and more flexible form.
Referrer-Policy	Limits how much of your URL is sent to third parties.	A privacy control and a mitigation for secrets accidentally placed in URLs — which should not happen and does.

Content-Security-Policy is the one worth the effort and the one that is hard. A strict policy requires eliminating inline script and inline event handlers, which is a real refactor on an existing application — so the honest advice is to deploy it in report-only mode first, read the reports for a month, and tighten incrementally, rather than recommending it flatly and watching it be abandoned.

- **Is every database query parameterised? Search the codebase for string concatenation or interpolation into SQL — it takes ten minutes and it is the single highest-value ten minutes in a web review, and it almost always finds at least one instance.**
- **Is every template auto-escaping, and is every escape bypass individually justified? `innerHTML`, `dangerouslySetInnerHTML`, `|safe`, `mark_safe`, and the equivalents are the complete greppable list.**
- **Is every state change a POST, with SameSite set and a token where it matters? And is there no state-changing GET endpoint anywhere — including in admin tools and legacy routes?**
- **Are the security headers set? Check with a scanner, and check every response rather than the home page — API responses and error pages frequently lack them.**
- **And is the CORS configuration as narrow as it can be? A wildcard origin combined with credentials is the specific finding to look for (§1), and it is a common result of making something work during development.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OWASP — the Cheat Sheet Series and the Top Ten (free)	The working reference for this entire module — specific, current, and defensive, with a sheet per class.
Zalewski — The Tangled Web	§1's model, explained properly — the best account of how the browser security model came to be what it is. Library copy; the author's Browser Security Handbook is free.
Google — CSP: A Successful Mess Between Hardening and Mitigation (free)	§4's honest assessment of CSP , from people who deployed it at scale — including why strict policies are hard.
MDN — Same-origin policy, CORS, and SameSite cookies (free)	The accurate current reference for §1 and §3 , which matters because the defaults have changed recently.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State the origin** of five URLs and say which pairs are same-origin.
- 2 **Demonstrate that a cross-origin request can be sent and not read**, on your own test pages.
- 3 **Find a permissive CORS configuration** in a training application and explain what it grants.
- 4 **Find and fix a SQL injection** in a deliberately vulnerable application, using parameterisation.

- 5 **Escape the same value for five different HTML contexts** and confirm the rules differ.
- 6 **Demonstrate that HTML escaping inside a script block is insufficient.**
- 7 **Build a CSRF demonstration** against your own test application, then fix it with SameSite and a token.
- 8 **Set every header in §4's table** on an application of yours and verify with a scanner.
- 9 **Deploy CSP in report-only mode** and read a week of reports.
- 10 **Run §4's five-point review** on a web application you own and report the findings.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define an origin and state what the same-origin policy blocks and does not.
- 2 Why does CSRF exist, given the policy?
- 3 What does CORS do, and why is the direction counterintuitive?
- 4 Give the single cause of every injection class and name six instances.
- 5 What is the defence that works, and why do the other three fail?
- 6 Why is input sanitisation structurally unable to be correct?
- 7 Why is one escapeHtml function insufficient for XSS?
- 8 Explain CSRF as a confused deputy and give the modern mitigations.
- 9 Name five security headers and what each does.
- 10 Why is CSP both the highest-value header and the hardest?